

Assignment 2 - Compilatori

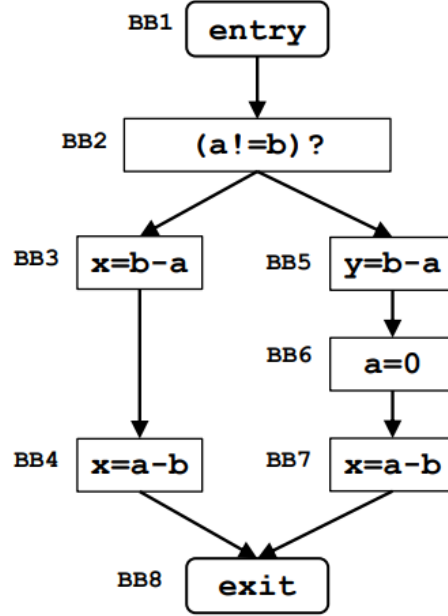
Emanuele Borghi (187264)

Gianluca Scollo (187575)

Lorenzo Benassi (187849)

14 Maggio 2026

1 Very Busy Expressions



Dataflow Problem: Very Busy Expressions	
Domain	Set of expressions
Direction	Backward: $IN[B] = f_B(OUT[B])$ $OUT[B] = \bigwedge IN(SUCC(B))$
Transfer Function	$IN[B] = Gen_B \cup (OUT[B] - Kill_B)$
Meet Operation ($\bigwedge$)	$\cap$
Boundary Condition	$OUT[Exit] = \emptyset$
Initial Interior Points	$OUT[B] = \cup$

Set of expressions: $\{b - a, a - b\} \rightarrow \langle bit_per_ (b - a), bit_per_ (a - b) \rangle$

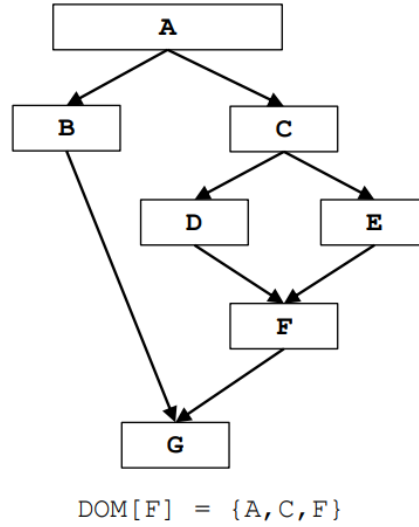
Basic Block	Iterazione 1	
	IN[B]	OUT[B]
BB8	$\langle 0, 0 \rangle$	$\langle 0, 0 \rangle$
BB7	$\langle 0, 1 \rangle$	$\langle 0, 0 \rangle$
BB6	$\langle 0, 0 \rangle$	$\langle 0, 1 \rangle$
BB5	$\langle 1, 0 \rangle$	$\langle 0, 0 \rangle$
BB4	$\langle 0, 1 \rangle$	$\langle 0, 0 \rangle$
BB3	$\langle 1, 1 \rangle$	$\langle 0, 1 \rangle$
BB2	$\langle 1, 0 \rangle$	$\langle 1, 0 \rangle$

Non avendo cicli all'interno del cfg abbiamo già raggiunto la convergenza con la prima iterazione

Procedimenti completi:

Kill[BB8]= $\{\emptyset\}$, Gen[BB8]= $\{\emptyset\}$, OUT[BB8]= $\{0,0\}$, IN[BB8]= $\{0,0\}$
 Kill[BB7]= $\{0,0\}$, Gen[BB7]= $\{0,1\}$, OUT[BB7]= $\{0,0\}$, IN[BB7]= $\{0,1\}$
 Kill[BB6]= $\{1,1\}$, Gen[BB6]= $\{0,0\}$, OUT[BB6]= $\{0,1\}$, IN[BB6]= $\{0,0\}$
 Kill[BB5]= $\{0,0\}$, Gen[BB5]= $\{1,0\}$, OUT[BB5]= $\{0,0\}$, IN[BB5]= $\{1,0\}$
 Kill[BB4]= $\{0,0\}$, Gen[BB4]= $\{0,1\}$, OUT[BB4]= $\{0,0\}$, IN[BB4]= $\{0,1\}$
 Kill[BB3]= $\{0,0\}$, Gen[BB3]= $\{1,0\}$, OUT[BB3]= $\{0,1\}$, IN[BB3]= $\{1,1\}$
 Kill[BB2]= $\{0,0\}$, Gen[BB2]= $\{0,0\}$, OUT[BB2]=(IN[BB3] $\cap$ IN[BB5])= $\{1,0\}$

2 Dominator Analysis

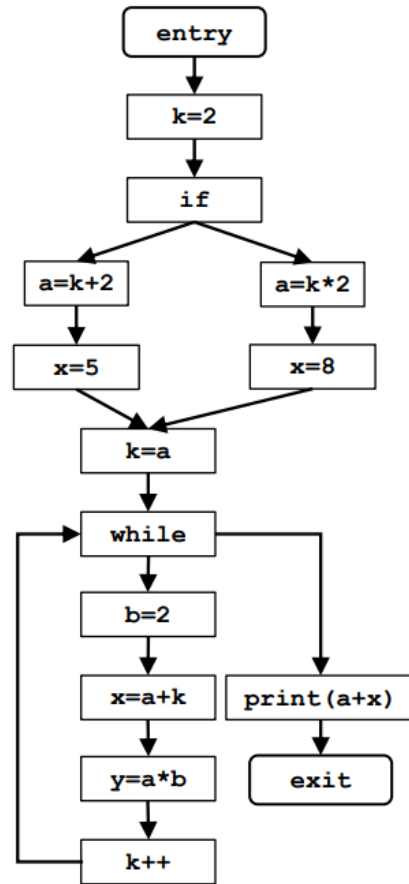


Dataflow Problem: Dominator Analysis	
Domain	Set of Basic Blocks
Direction	Forward: $OUT[B] = f_B(IN[B])$ $IN[B] = \bigwedge OUT(PRED(B))$
Transfer Function	$OUT[B] = GEN[B] \cup IN[B]$
Meet Operation ($\wedge$)	$\cap$
Boundary Condition	$IN[Entry] = \emptyset$
Initial Interior Points	$OUT[B] = \cup$

Partendo dall'entry block, per calcolare l'IN di ogni block è necessario intersecare tra loro gli OUT dei blocchi padre ed è necessario che inserisca sé stesso all'interno del suo output.

Basic Block	Iterazione 1	
	IN[B]	OUT[B]
A	$\langle \emptyset \rangle$	$\langle A \rangle$
B	$\langle A \rangle$	$\langle A, B \rangle$
C	$\langle A \rangle$	$\langle A, C \rangle$
D	$\langle A, C \rangle$	$\langle A, C, D \rangle$
E	$\langle A, C \rangle$	$\langle A, C, E \rangle$
F	$\text{OUT}[D] \cap \text{OUT}[E] = \langle A, C \rangle$	$\langle A, C, F \rangle$
G	$\text{OUT}[F] \cap \text{OUT}[B] = \langle A \rangle$	$\langle A, G \rangle$

3 Constant Propagation



Dataflow Problem: Constant Propagation	
Domain	Mappatura delle variabili sui valori del reticolo: Variabili $\rightarrow \{\perp, c\}$
Direction	Forward: OUT[B] = f_B (IN[B]) IN[B] = $\bigwedge$ OUT(PRED(B))
Transfer Function	Valuta ogni istruzione aggiornando il valore della variabile nel reticolo: • $x = c \rightarrow$ assegna c a x • $x = y + z \rightarrow$ calcola (IN(B)(y) + IN(B)(z)) usando le regole del reticolo • Se l'istruzione usa un valore non costante, il risultato per quella variabile è $\perp$ (valore non noto)
Meet Operation ($\bigwedge$)	$\sqcap$ (Operatore di meet del reticolo delle costanti) Regole di intersezione sui bivi per ogni variabile x : • $c \sqcap c = c$ • $c_1 \sqcap c_2 = \perp$ (se $c_1 \neq c_2$) • $x \sqcap \perp = \perp$
Boundary Condition	IN[Entry] = $\emptyset$ (non ho conoscenza)
Initial Interior Points	OUT[B] = $\cup$

Set of variables: $\{a, b, k, x, y\} \rightarrow$ ogni variabile può assumere due possibili valori: $\perp$ (il valore non è noto) c (valore numerico)

Basic Block	Iterazione 1		Iterazione 2	
	IN[B]	OUT[B]	IN[B]	OUT[B]
BB1 (entry)	$\perp, \perp, \perp, \perp, \perp$	$\perp, \perp, \perp, \perp, \perp$	$\perp, \perp, \perp, \perp, \perp$	$\perp, \perp, \perp, \perp, \perp$
BB2 (k=2)	$\perp, \perp, \perp, \perp, \perp$	$\perp, \perp, 2, \perp, \perp$	$\perp, \perp, \perp, \perp, \perp$	$\perp, \perp, 2, \perp, \perp$
BB3 (if)	$\perp, \perp, 2, \perp, \perp$	$\perp, \perp, 2, \perp, \perp$	$\perp, \perp, 2, \perp, \perp$	$\perp, \perp, 2, \perp, \perp$
BB4 (a=k+2)	$\perp, \perp, 2, \perp, \perp$	$4, \perp, 2, \perp, \perp$	$\perp, \perp, 2, \perp, \perp$	$4, \perp, 2, \perp, \perp$
BB5 (x=5)	$4, \perp, 2, \perp, \perp$	$4, \perp, 2, 5, \perp$	$4, \perp, 2, \perp, \perp$	$4, \perp, 2, 5, \perp$
BB6 (a=k*2)	$\perp, \perp, 2, \perp, \perp$	$4, \perp, 2, \perp, \perp$	$\perp, \perp, 2, \perp, \perp$	$4, \perp, 2, \perp, \perp$
BB7 (x=8)	$4, \perp, 2, \perp, \perp$	$4, \perp, 2, 8, \perp$	$4, \perp, 2, \perp, \perp$	$4, \perp, 2, 8, \perp$
BB8 (k=a) (Meet BB5 e BB7)	$4, \perp, 2, \perp, \perp$	$4, \perp, 4, \perp, \perp$	$4, \perp, 2, \perp, \perp$	$4, \perp, 4, \perp, \perp$
BB9 (while) (Meet BB9 e BB13)	$4, \perp, 4, \perp, \perp$	$4, \perp, 4, \perp, \perp$	$4, \perp, \perp, \perp, \perp$	$4, \perp, \perp, \perp, \perp$
BB10 (b=2)	$4, \perp, 4, \perp, \perp$	$4, 2, 4, \perp, \perp$	$4, \perp, \perp, \perp, \perp$	$4, 2, \perp, \perp, \perp$
BB11 (x=a+k)	$4, 2, 4, \perp, \perp$	$4, 2, 4, 8, \perp$	$4, 2, \perp, \perp, \perp$	$4, 2, \perp, \perp, \perp$
BB12 (y=a*b)	$4, 2, 4, 8, \perp$	$4, 2, 4, 8, 8$	$4, 2, \perp, \perp, \perp$	$4, 2, \perp, \perp, 8$
BB13 (k++)	$4, 2, 4, 8, 8$	$4, 2, 5, 8, 8$	$4, 2, \perp, \perp, 8$	$4, 2, \perp, \perp, 8$
BB14 (print(a+x))	$4, \perp, 4, \perp, \perp$	$4, \perp, 4, \perp, \perp$	$4, \perp, \perp, \perp, \perp$	$4, \perp, \perp, \perp, \perp$

Alla prima iterazione si può sapere quanto vale k , ma dalla seconda iterazione dopo l'esecuzione del Meet nel while non posso più conoscere il valore di k perché è cambiato, di conseguenza, sempre nell'iterazione 2, al più possiamo conoscere il valore di b e di y .

Le successive iterazioni saranno uguali all'iterazione 2.

Per qualunque iterazione non sarà quindi possibile avere una constant propagation che permetta di avere un valore costante come valore per la print da stampare